9.4. Strategy

The Strategy pattern is used to decouple a method from an object, allowing you to supply many possible instances of the method. Our discussion of the Strategy pattern illustrates a structuring technique found in many object-oriented programs, that of *parallel class hierarchies*.We will illustrate the Strategy pattern by considering how tax payers may apply different tax strategies. There will be a hierarchy for tax payers, and a related hierarchy for tax strategies. For example, there is a default strategy that applies to any tax payer. One subclass of tax payer is a trust, and one subclass of the default strategy is one that applies only to trusts.

Our discussion will also illustrate a technique often used with generic types—the use of type variables with *recursive bounds*.We have already seen this trick at work in the definition of the `Comparable` interface and the `Enum` class; here we will apply it to clarify the connection between tax payers and their associated tax strategies. We also explain the `getThis` trick, which allows us to assign a more precise type to `this` when type variables with recursive bounds appear.

First, we'll look at a basic version of the Strategy pattern, which shows how to use generics to design parallel class hierarchies. Next, we'll look at an advanced version where objects contain their own strategies, which uses type variables with recursive bounds and explains the `getThis` trick.

The example in this section was developed in discussion with Heinz M. Kabutz, and also appears in his online publication, *The Java Specialists' Newsletter*.

Parallel Class Hierarchies A typical use of the Strategy pattern is for tax computation, as shown in Example 9-6. We have a class `TaxPayer` with subclasses `Person` and `Trust`. Every tax payer has an income, and, in addition, a trust may be nonprofit. For example, we

Example 9-6. A basic Strategy pattern with parallel class hierarchies

```
abstract class TaxPayer {
  public long income; // in cents
  public TaxPayer(long income) { this.income = income; }
  public long getIncome() { return income; }
}
class Person extends TaxPayer {
  public Person(long income) { super(income); }
}
class Trust extends TaxPayer {
  private boolean nonProfit;
  public Trust(long income, boolean nonProfit) {
    super(income); this.nonProfit = nonProfit;
  }
  public boolean isNonProfit() { return nonProfit; }
}

interface TaxStrategy<P extends TaxPayer> {
  public long computeTax(P p);
}
class DefaultTaxStrategy<P extends TaxPayer> implements TaxStrategy<P> {
  private static final double RATE = 0.40;
```

```
  public long computeTax(P payer) {
    return Math.round(payer.getIncome() * RATE);
  }
}
class DodgingTaxStrategy<P extends TaxPayer> implements TaxStrategy<P> {
  public long computeTax(P payer) { return 0; }
}
class TrustTaxStrategy extends DefaultTaxStrategy<Trust> {
  public long computeTax(Trust trust) {
    return trust.isNonProfit() ? 0 : super.computeTax(trust);
  }
}
```

may define a person with an income of $100,000.00 and two trusts with the same income, one nonprofit and one otherwise:

```
Person person = new Person(10000000);
Trust nonProfit = new Trust(10000000, true);
Trust forProfit = new Trust(10000000, false);
```

In accordance with good practice, we represent all monetary values, such as incomes or taxes, by long integers standing for the value in cents (see the item “Avoid float and double if exact answers are required”, in the General Programming chapter of *Effective Java* by Joshua Bloch, Addison-Wesley).

For each tax payer `P` there may be many possible strategies for computing tax. Each strategy implements the interface `TaxStrategy<P>`, which specifies a method `computeTax` that takes as argument a tax payer of type `P` and returns the tax paid. Class `DefaultTaxStrategy` computes the tax by multiplying the income by a fixed tax rate of 40 percent, while class `DodgingTaxStrategy` always computes a tax of zero:

```
TaxStrategy<Person> defaultStrategy = new DefaultStrategy<Person>();
TaxStrategy<Person> dodgingStrategy = new DodgingStrategy<Person>();
assert defaultStrategy.computeTax(person) == 4000000;
assert dodgingStrategy.computeTax(person) == 0;
```

Of course, our example is simplified for purposes of illustration—we do not recommend that you compute taxes using either of these strategies! But it should be clear how these techniques extend to more complex tax payers and tax strategies.

Finally, class `TrustTaxStrategy` computes a tax of zero if the trust is nonprofit and uses the default tax strategy otherwise:

```
TaxStrategy<Trust> trustStrategy = new TrustTaxStrategy();
assert trustStrategy.computeTax(nonProfit) == 0;
assert trustStrategy.computeTax(forProfit) == 4000000;
```

Generics allow us to specialize a given tax strategy to a given type of tax payer, and allow the compiler to detect when a tax strategy is applied to the wrong type of tax payer:

```
trustStrategy.computeTax(person); // compile-time error
```

Without generics, the `computeTax` method of `TrustTaxStrategy` would have to accept an argument of type `TaxPayer` and cast it to type `Trust`, and errors would throw an exception at run time rather than be caught at compile time.

This example illustrates a structuring technique found in many object-oriented programs—that of *parallel class hierarchies*. In this case, one class hierarchy consists of `TaxPayer`, `Person`, and `Trust`. A parallel class hierarchy consists of strategies corresponding to each of these: two strategies, `DefaultTaxStrategy` and `DodgingTaxStrategy` apply to any `TaxPayer`, no specialized strategies apply to `Person`, and there is one specialized strategy for `Trust`.

Usually, there is some connection between such parallel hierarchies. In this case, the `computeTax` method for a `TaxStrategy` that is parallel to a given `TaxPayer` expects an argument that is of the corresponding type; for instance, the `computeTax` method for `TrustTaxStrategy` expects an argument of type `Trust`. With generics, we can capture this connection neatly in the types themselves. In this case, the `computeTax` method for `TaxStrategy<P>` expects an argument of type `P`, where `P` must be subclass of `TaxPayer`. Using the techniques we have described here, generics can often be used to capture similar relations in other parallel class hierarchies.

An Advanced Strategy Pattern with Recursive Generics In more advanced uses of the Strategy pattern, an object contains the strategy to be applied to it. Modelling this situation requires recursive generic types and exploits a trick to assign a generic type to `this`.

The revised Strategy pattern is shown in Example 9-7. In the advanced version, each tax payer object contains its own tax strategy, and the constructor for each kind of tax payer includes a tax strategy as an additional argument:

```
Person normal = new Person(10000000, new DefaultTaxStrategy<Person>());
Person dodger = new Person(10000000, new DodgingTaxStrategy<Person>());
Trust nonProfit = new Trust(10000000, true, new TrustTaxStrategy());
Trust forProfit = new Trust(10000000, false, new TrustTaxStrategy());
```

Now we may invoke a `computeTax` method of no arguments directly on the tax payer, which will in turn invoke the `computeTax` method of the tax strategy, passing it the tax payer:

```
assert normal.computeTax() == 4000000;
assert dodger.computeTax() == 0;
assert nonProfit.computeTax() == 0;
assert forProfit.computeTax() == 4000000;
```

This structure is often preferable, since one may associate a given tax strategy directly with a given tax payer.

Before, we used a class `TaxPayer` and an interface `TaxStrategy<P>`, where the type variable `P` stands for the subclass of `TaxPayer` to which the strategy applies. Now we must add the type parameter `P` to both, in order that the class `TaxPayer<P>` can have a field of type `TaxStrategy<P>`. The new declaration for the type variable `P` is necessarily recursive, as seen in the new header for the `TaxPayer` class:

```
class TaxPayer<P extends TaxPayer<P>>
```

We have seen similar recursive headers before:

```
interface Comparable<T extends Comparable<T>>
class Enum<E extends Enum<E>>
```

In all three cases, the class or interface is the base class of a type hierarchy, and the type parameter stands for a specific subclass of the base class. Thus, `P` in `TaxPayer<P>` stands for the specific kind of tax payer, such as `Person` or `Trust`; just as `T` in `Comparable<T>` stands for the specific class being compared, such as `String`; or `E` in `Enum<E>` stands for the specific enumerated type, such as `Season`.

The tax payer class contains a field for the tax strategy and a method that passes the tax payer to the tax

strategy, as well as a recursive declaration for P just like the one used in TaxPayer. In outline, we might expect it to look like this:

```
// not well-typed!
class TaxPayer<P extends TaxPayer<P>> {
  private TaxStrategy<P> strategy;
  public long computeTax() { return strategy.computeTax(this); }
  ...
}
class Person extends TaxPayer<Person> { ... }
class Trust extends TaxPayer<Trust> { ... }
```

But the compiler rejects the above with a type error. The problem is that this has type TaxPayer<P>, whereas the argument to computeTax must have type P. Indeed, within each specific tax payer class, such as Person or Trust, it is the case that this does have type P; for example, Person extends TaxPayer<Person>, so P is the same as Person within this class. So, in fact, this will have the same type as P, but the type system does not know that!

We can fix this problem with a trick. In the base class TaxPayer<P> we define an abstract method getThis that is intended to return the same value as this but gives it the type P. The method is instantiated in each class that corresponds to a specific kind of tax payer, such as Person or Trust, where the type of this is indeed the same as the type P. In outline, the corrected code now looks like this:

```
// now correctly typed
abstract class TaxPayer<P extends TaxPayer<P>> {
  private TaxStrategy<P> strategy;
  protected abstract P getThis();
  public long computeTax() { return strategy.computeTax(getThis()); }
  ...
}
final class Person extends TaxPayer<Person> {
  protected Person getThis() { return this; }
  ...
}
final class Trust extends TaxPayer<Trust> {
  protected Trust getThis() { return this; }
  ...
}
```

The differences from the previous code are in bold. Occurences of this are replaced by calls to getThis; the method getThis is declared abstract in the base class and it is instantiated appropriately in each final subclass of the base class. The base class TaxPayer<P> must be declared abstract because it declares the type for getThis but doesn't declare the body. The body for getThis is declared in the final subclasses Person and Trust.

Since Trust is declared final, it cannot have subclasses. Say we wanted a subclass NonProfitTrust of Trust. Then not only would we have to remove the final declaration on the class Trust, we would also need to add a type parameter to it. Here is a sketch of the required code:

```
abstract class Trust<T extends Trust<T>> extends TaxPayer<T> { ... }
final class NonProfitTrust extends Trust<NonProfitTrust> { ... }
final class ForProfitTrust extends Trust<ForProfitTrust> { ... }
```

Example 9-7. An advanced Strategy pattern with recursive bounds

```
abstract class TaxPayer<P extends TaxPayer<P>> {
  public long income; // in cents
  private TaxStrategy<P> strategy;
  public TaxPayer(long income, TaxStrategy<P> strategy) {
    this.income = income; this.strategy = strategy;
  }
  protected abstract P getThis();
  public long getIncome() { return income; }
  public long computeTax() { return strategy.computeTax(getThis()); }
}
class Person extends TaxPayer<Person> {
  public Person(long income, TaxStrategy<Person> strategy) {
    super(income, strategy);
  }
  protected Person getThis() { return this; }
}
class Trust extends TaxPayer<Trust> {
  private boolean nonProfit;
  public Trust(long income, boolean nonProfit, TaxStrategy<Trust> strategy){
    super(income, strategy); this.nonProfit = nonProfit;
  }
  protected Trust getThis() { return this; }
  public boolean isNonProfit() { return nonProfit; }
}

interface TaxStrategy<P extends TaxPayer<P>> {
  public long computeTax(P p);
}
class DefaultTaxStrategy<P extends TaxPayer<P>> implements TaxStrategy<P> {
  private static final double RATE = 0.40;
  public long computeTax(P payer) {
    return Math.round(payer.getIncome() * RATE);
  }
}
class DodgingTaxStrategy<P extends TaxPayer<P>> implements TaxStrategy<P> {
  public long computeTax(P payer) { return 0; }
}
class TrustTaxStrategy extends DefaultTaxStrategy<Trust> {
  public long computeTax(Trust trust) {
    return trust.isNonprofit() ? 0 : super.computeTax(trust);
  }
}
```

Now an instance of `NonProfitTrust` takes a strategy that expects a `NonProfitTrust` as an argument, and `ForProfitTrust` behaves similarly. It is often convenient to set up a parameterized type hierarchy in this way, where classes with subclasses take a type parameter and are `abstract` and classes without subclasses do not take a type parameter and are `final`. A body for the `getThis` method is declared in each `final` subclass.

Summary As we have seen, recursive type parameters often appear in Java:

```
class TaxPayer<P extends TaxPayer<P>>
Comparable<T extends Comparable<T>>
class Enum<E extends Enum<E>>
```

The `getThis` trick is useful in this situation whenever one wants to use `this` in the base type with the more specific type provided by the type parameter.

We will see another example of recursive type parameters in the next section, applied to two mutually recursive classes. However, although the `getThis` trick is often useful, we will not require it there.